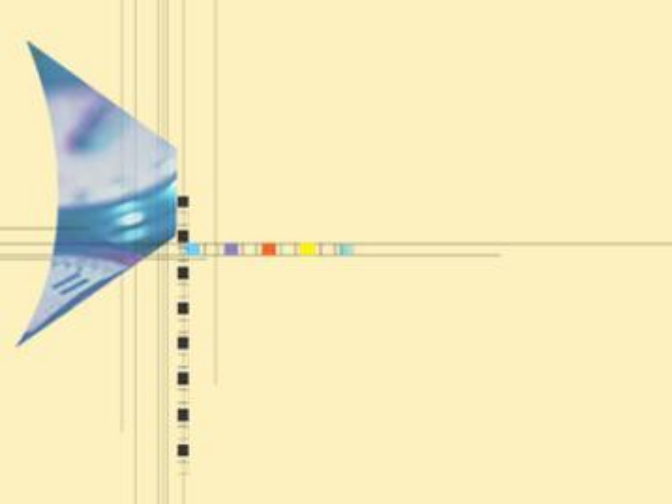




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

主要内容



1.1 算法的基本概念

1.2 算法分析基础

1.3 算法时间估算方法

1.4 求解递推关系



主要内容



1.2 算法分析基础

- 算法分析的目的
- 算法分析的基本方法
- 时间复杂性
- 渐近表示符号
- 空间复杂性
- 什么是最优算法
- 不同情况算法复杂性



1.2.1 算法分析的目的

- 算法分析是对一个算法需要多少计算时间和存储空间作定量的分析。一般计算出相应的数量级，常用时间复杂度和空间复杂度表示。
- 目的是评价算法的效率以求改进。要降低算法的时间复杂度和空间复杂度，提高算法的执行效率。

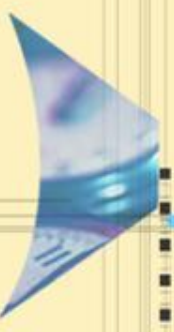
算法的评价

- 在实践中需要选择有用的算法。一个有用的算法不仅仅是要求执行步骤有穷，而且要求执行步骤相当有穷（在合理的步骤后停机）
- 对于同一个问题，可能存在多个算法。那么那个算法好一点呢？
 - 解决问题所需要的时间和空间
 - 其它一些所需要的资源，例如通讯开销，处理器的数目，等等

1.2.2 算法分析的基本方法

- **事前分析**：就算法本身，通过对其执行性能的理论分析，得出关于算法特性——时间和空间的一个特征函数（ O 、 Ω ）——与计算机软硬件没有直接关系。
- **事后测试**：将算法编制成程序后放到计算机上运行，收集其执行时间和空间占用等统计资料，进行分析判断——直接与物理实现有关。





例1: 搜索问题 (查找问题)

- 搜索问题
- 顺序查找
- 二分查找



例1: 搜索问题

- 假定元素取自于线序集合，例如整数集。在一些类似的问题中，如查找中项、第 k 个最小元素等的问题中也做此假设。
- 问题描述（设定数据结构）
 - 设 $A[1..n]$ 为一个 n 个元素的数组，判定给定元素 x 是否在 A 中。
 - 这个问题也可以这样表述:寻找是否满足有 $x=A[j]$ 的元素索引 j ($1 \leq j \leq n$)，如果 x 不在 A 中， $j=0$ 。

算法1: Sequential search (linear search)顺序查找



基本思想

- 扫描A中的所有项目，将每个项目与x比较；
- 如果在j次比较后 $1 \leq j \leq n$ 搜索成功，即 $x=A[j]$ ，则返回j的值；
- 否则返回0，表示没有找到。



算法1.1 LINEARSEARCH

输入: n 个元素的数组 $A[1 \cdots n]$ 和元素 x 。

输出: 如果 $x = A[j]$, $1 \leq j \leq n$, 则输出 j , 否则输出 0.

1. $j \leftarrow 1$
2. **while** ($j < n$) **and** ($x \neq A[j]$)
3. $j \leftarrow j + 1$
4. **end while**
5. **if** $x = A[j]$ **then return** j **else return** 0.



算法2: Binary search (二分查找)



基本思想

- 令 $A[low \cdots high]$ 为元素按升序排列的非空数组， $A[mid]$ 为中间元素，假定 $x > A[mid]$;
- 注意到如果 x 在 A 中，则它必定是 $A[mid+1], A[mid+2], \dots, A[high]$ 中的一个;
- 接下来只需在 $A[mid+1 \cdots high]$ 中搜索 x 。



算法1.2 BINARYSEARCH

输入: n 个元素的升序数组 $A[1 \cdots n]$ 和元素 x 。

输出: 如果 $x = A[j]$, $1 \leq j \leq n$, 则输出 j , 否则输出 0。

1. $low \leftarrow 1; high \leftarrow n; j \leftarrow 0$

2. **while** ($low \leq high$) **and** ($j = 0$)

3. $mid \leftarrow \lfloor low + high / 2 \rfloor$

4. **if** $x = A[mid]$ **then** $j \leftarrow mid$

5. **else if** $x < A[mid]$ **then** $high \leftarrow mid - 1$

6. **else** $low \leftarrow mid + 1$

7. **end while**

8. **return** j



算法1.1和1.2 中元素比较操作的执行次数

假设数组大小为 n

算法1.1: 元素比较操作最多执行 n 次。

算法1.2:

定理1.1 对于一个大小为 n 的排好序的数组，算法
BINARYSEARCH执行元素比较的最大次数为 $\lfloor \log n \rfloor + 1$ 。
(见教材)

当 n 很大时，哪个算法效率更高？



例2: 排序问题

问题描述:

令 $A[1..n]$ 为一个有 n 个元素的数组，通过数组元素之间的比较作为基本操作，将 A 中元素进行排序。

- 排序算法：
 - 选择排序
 - 插入排序
 - 自底向上排序

例2: 选择排序 (Selection sort)

基本思想

令 $A[1..n]$ 为一个有 n 个元素的数组

- 首先找到最小元素，将其存放在 $A[1]$ 中；
- 然后找到剩下的 $n-1$ 个元素中的最小元素，将其存放在 $A[2]$ 中；
- 重复此过程直至找到第二大的元素，并将其存放在 $A[n-1]$ 中

算法1.4 SELECTIONSORT

输入: n 个元素的数组 $A[1..n]$ 。

输出: 按非降序排列的数组 $A[1..n]$

1. **for** $i \leftarrow 1$ **to** $n-1$
2. $k \leftarrow i$
3. **for** $j \leftarrow i+1$ **to** n {查找第 i 小的元素}
4. **if** $A[j] < A[k]$ **then** $k \leftarrow j$
5. **end for**
6. **if** $k \neq i$ **then** 交换 $A[i]$ and $A[k]$
7. **end for**



主要操作执行次数分析

观察结论1.3

- 执行算法SELECTIONSORT所需的元素比较次数为 $n(n-1)/2$ 。

$$\sum_{i=1}^{n-1} (n-i) = (n-1) + (n-2) + \cdots + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

- 执行算法SELECTIONSORT所需的元素赋值次数为 $0 \sim 3(n-1)$ 之间。

插入排序 (Insertion sort)

基本思想

- 依次将第2个到最后一个元素进行插入。但选定一个要插入元素时，它前面的元素已经是排好序的。
- 最初，只含有元素 $A[1]$ 的大小为1的子数组是有序的，从 $A[2]$ 开始依次插入元素。将 $A[2]$ 插入到 $A[1]$ 的前面或后面，这取决于 $A[2]$ 比 $A[1]$ 小还是比 $A[1]$ 大。
- 继续这一过程，那么在第 i 次执行中，要将 $A[i]$ 插入到已排序的子数组 $A[1..i-1]$ 中的合适位置上。应这样进行：依次扫描序号从 $i-1$ 到1的元素，每次都把 $A[i]$ 和当前位置的元素比较。
- 在扫描的每一步，元素都被移到序号更高大的一个位置，这种执行比较和移位的扫描过程直到以下情况出现时为止：或者找到一个小于等于 $A[i]$ 的元素，或者前面已排序数组的元素都已扫描过。在这种情况下，已找到 $A[i]$ 被插的合适位置，插入 $A[i]$ 的过程就完成了。

算法1.5 INSERTIONSORT

输入: n 个元素的数组 $A[1..n]$

输出:按非降序排列的数组 $A[1..n]$

```
1. for  $i \leftarrow 2$  to  $n$ 
2.    $x \leftarrow A[i]$ 
3.    $j \leftarrow i-1$ 
4.   while  $(j > 0)$  and  $(A[j] > x)$ 
5.      $A[j+1] \leftarrow A[j]$  // 移位
6.      $j \leftarrow j-1$ 
7.   end while
8.    $A[j+1] = x$ 
9. end for
```

➤ i : 待插入元素的下标;

➤ j : 在前面已排序队列中向前移位的下标;

➤ x : 用于临时存储, 进行交换时的临时存储空间, 主要用来存储 $A[i]$ 数据;

➤for循环: 每次取一个新元素, 进行插入操作。

➤while循环: 寻找要插入元素的插入位置。具体是将它 (初始在第 i 个位置) 依次同它前的元素比较 (它之前的元素都已排好序), 比它大的元素依次后移一位。直到不再大于它为止。循环结束后, 第 i 个元素要插入的位置为 $j+1$;

主要操作执行次数分析

观察结论1.4

- 执行算法INSERTIONSORT的元素比较次数在 $n-1$ 到 $n(n-1)/2$ 之间。元素赋值次数等于元素比较次数加上 $n-1$ 。

自底向上合并排序 (Bottom-Up merge sorting)

基本思想:

- 初始时, 将数组中每个元素看作是已经按要求排好序的子数组, 一共有 n 个大小为1的排好序的子数组;
- 将 n 个排好序的子数组两两合并成大小为2的 $n/2$ 个排好序的数组;
- 再将 $n/2$ 个大小为2的排好序的数组两两合并成 $n/4$ 个大小为4的排好序的数组;
- 依次进行合并下去, 最终得到大小为 n 的排好的数组。
- 从这个排序过程来看, 排序是从将长度为 1 的有序的子数组开始, 逐步合并变化到长度为 n 的有序数组, 因而是**自底向上**的。

算法1.6 BOTTOMUPSORT

输入: n 个元素的数组 $A[1..n]$

输出: 按非降序排列的数组 $A[1..n]$

1. $t \leftarrow 1$

2. **while** $t < n$

3. $s \leftarrow t$; $t \leftarrow 2s$; $i \leftarrow 0$

4. **while** $i + t \leq n$

5. MERGE($A, i+1, i + s, i+t$)

6. $i \leftarrow i+t$

7. **end while**

8. **if** $i + s < n$ **then** MERGE($A, i+1, i + s, n$)

9. **end while**

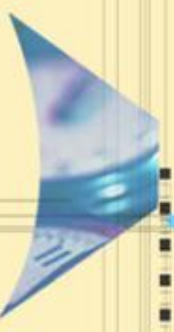
➤ t : 每次合并后数组元素个数,
➤ s : 每次合并前数组元素个数,
 i : 跳动指针;

➤ 外层while循环分层进行排好序的数组的合并。合并前子数组大小为 s , 合并后 t . 合并后 s 被乘以2;

➤ 内层while循环表示同层的若干过排好序的数组的两两合并;
 $i+1, i + s$ and $i+t$ 用来定义两个要合并的数组的边界。

➤ $i \leftarrow i+t$ 表示每次往后移动 t 个单元, 得到下一对要合并的数组的起点边界;





关键过程：合并两个子序列 (Merging two sorted lists)

问题描述

- 将两个排好序的数组合并成一个更大的排好序的数组。
- 假定有一个数组 $A[1 \cdots m]$ ， p, q, r 为它的三个索引，并有 $1 \leq p \leq q < r \leq m$ ，两个子数组 $A[p \cdots q]$ 和 $A[q+1 \cdots r]$ 各自按升序排列，重新安排 A 中元素的位置，使得子数组 $A[p \cdots r]$ 也是按升序排列。

Merge基本思想

- 使用两个指针s和t，初始化时各自指向和合并的两个子数组起始位置： $A[p]$ 和 $A[q+1]$ 。额外创建一个辅助数组 $B[p..r]$ ，用作合并过程的做暂存器，指针k指向B中当前可存元素的下标。
- 每一次比较元素 $A[s]$ 和 $A[t]$ ，将小的元素添加到辅助数组B，更新B中元素下标指针：k加1。
- 然后更新指针s或者t，如果 $A[s] \leq A[t]$ ，则s加1，否则t加1，
- 当条件 $s=q+1$ 或 $t=r+1$ 成立时过程结束。在第一种情况下，我们把数组 $A[t..r]$ 中剩余的元素添加到B，而在另一种情况下，把数组 $A[s..q]$ 中剩余的元素添加到B。
- 最后，把数组 $B[p..r]$ 复制回 $A[p..r]$ 。

算法1.3 MERGE

输入: 数组 $A[1..m]$ 和它的三个索引 p, q, r , $1 \leq p \leq q < r \leq m$, 两个子数组 $A[p..q]$ 和 $A[q+1..r]$ 各自按升序排列。

输出: 合并两个子数组 $A[p..q]$ 和 $A[q+1..r]$ 的数组 $A[p..r]$ 。

```
1. comment:  $B[p..r]$  是个辅助数组
2.  $s \leftarrow p; t \leftarrow q + 1; k \leftarrow p$ 
3. while  $s \leq q$  and  $t \leq r$ 
4.   if  $A[s] \leq A[t]$  then
5.      $B[k] \leftarrow A[s]; s \leftarrow s + 1$ 
6.   else  $B[k] \leftarrow A[t]; t \leftarrow t + 1$ 
7.   end if
8.    $k \leftarrow k + 1$ 
9. end while
10. if  $s = q + 1$  then  $B[k..r] \leftarrow A[t..r]$ 
11. else  $B[k..r] \leftarrow A[s..q]$ 
12. end if
13.  $A[p..r] \leftarrow B[p..r]$ 
```

- s 是 $A[p, \dots, q]$ 数组指针;
- t 是 $A[q+1, \dots, r]$ 数组指针;
- k 是 $B[p, \dots, r]$ 数组指针

Merge中主要操作次数算法分析

观察结论1.1

- 执行算法MERGE将两个大小分别为 n_1 和 n_2 的非空数组合并成一个大小为 $n=n_1+n_2$ 的排序数组，当 $n_1 \leq n_2$ 时，元素比较次数在 n_1 到 $n-1$ 之间。特例是，如果两个数组大小为 $\lfloor n/2 \rfloor$ 和 $\lceil n/2 \rceil$ ，需要比较的次数在 $\lfloor n/2 \rfloor$ 到 $n-1$ 之间

观察结论1.2

- 使用算法MERGE将两个数组合并为一个大小为 n 的有序数组，元素赋值的次数恰好是 $2n$

BOTTOMUPSORT中主要操作次数分析

- 当 n 是2的幂情况下，外部的while循环的执行次数是 $k=\log n$ 次，如 $n=8$ ，则 $k=3$ 次外while循环，3层树结构。
- 通常，在第 j 次迭代中，对于大小为 2^{j-1} 的数组合并(MERGE)操作次数是 $n/2^j$ 。如：第一次迭代， $j=1$ ， $n=10$ ，则合并(MERGE)操作次数， $10/2=5$ 次。

观察结论1.5

- 总共的元素比较次数是 $(n \log n) / 2$ 和 $n \log n - n + 1$ 个之间。其中 n 是2的幂；元素赋值总数是 $2n \log n$ 。

三个算法哪个算法效率高呢?

- SELECTIONSORT和INSERTIONSORT算法, n 个元素进行排序所需的最多元素比较操作次数与 n^2 成正比增长。元素赋值操作于 n 成正比增长。
- BOTTOMUPSORT算法, n 个元素进行排序所需的最多元素比较操作次数与 $n\log n$ 成正比增长。元素赋值与 $n\log n$ 成正比增长。
- 当 n 很大时, 哪个算法效率更高呢?

元运算/元操作(elementary operation)

定义1.1. 对于任何一个计算步骤，如果它的一次执行代价总是以一个时间常数为上界，与输入数据或者使用何种算法无关，则称该计算步骤为“元运算”或“元操作”。

例：

算术运算： $+$, $-$, $*$, $/$

比较和逻辑运算： $=$, $\&$, $|$, $\sim$

赋值运算：包括遍历表或树的指针赋值



频度

频度：算法中某个语句或运算的执行次数称为其频度。

例：

$x \leftarrow x+y$

(a)

(a): $x=x+y$ 执行了1次

for $i \leftarrow 1$ to n

$x \leftarrow x+y$

end for

(b)

(b): $x=x+y$ 执行了 n 次

for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to n

$x \leftarrow x+y;$

end for

end for

(c)

(c): $x=x+y$ 执行了 n^2 次



算法的运行时间函数

- 由于元操作的执行时间以一个时间常数为上界，因此就可以确定算法中加、减、乘、除、比较、读、写等基本操作所需的时间，从而得到算法总的运行时间 t_p 的公式。
- 令 n 代表算法的输入大小（这里主要是问题规模），则算法运行时间可表示为如下公式：

$$t(n) = caADD(n) + csSUB(n) + cmMUL(n) + cdDIV(n) + ccCMP(n) + \dots$$

其中， ca ， cs ， cm ， cd ， cc 分别表示一次加、减、乘、除及比较操作等所需的时间，函数 ADD ， SUB ， MUL ， DIV ， CMP 分别表示算法中所使用的加、减、乘、除及比较操作的次数。

算法的运行时间函数增长趋势分析

- 实际上，我们不需要精确的时间来评估算法的运行效率，我们感兴趣的是估计输入数据量 n 很大情况下算法的运行时间。因此，通常我们使用运行时间的增长速率和或增长趋势来评估效率。当 n 越来越大时，函数低阶项和产量对函数的变化趋势影响越来越弱最后趋于0。
- 例： $f(n)=n^2\log n+10n^2+n$
 $f(n)$ 具有同最高阶项 $n^2\log n$ 同阶的增长趋势。
- 可见，算法的运行时间函数中的最高阶项是决定算法运行时间增长趋势的关键因素，对应执行次数最多的关键操作。

基本运算 (basic operation)

定义 1.6. 如果算法中的一个元运算具有最高频度，所有其他元运算频度均在它的频度的常数倍内，则称这个元运算为基本运算。

可见，在分析一个算法的运行时间时，只需挑选出一个频度至少和任何其他任何运算的频度一样大的**基本运算**。这个关键运算对算法时间影响最大。统计基本运算的频度就可以。

例1: 寻找最大元素

算法 Max

输入: n 个元素的数组 $A[1..n]$

输出: $pos, A[pos]$ 为数组中的最大元素

comment:寻找 $A[1..n]$ 中的最大元素

1. $pos \leftarrow 1$
2. for $i \leftarrow 2$ to n
3. if $A[pos] < A[i]$
4. $pos \leftarrow i$
5. end if
6. end for



例1: 寻找最大元素

- 基本运算: 元素的比较。
- for 循环中共进行了 $n-1$ 次比较: for 循环每次执行之前都要比较一下 $A[pos]$ 和 $A[i]$
- Max 还进行了其它的操作, 如初始化 pos 、循环控制变量 i 的增量。循环内 pos 的赋值等。这些操作的频度都没有元素比较操作频度高。



例2: 计算 $A[1..n]$ 中元素的排名

算法 RANK

输入: n 个元素的数组 $A[1..n]$

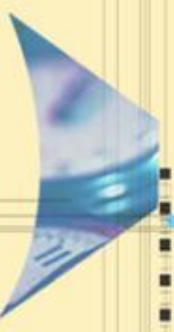
输出: 数组 $R[1..n]$, $R[i]$ 为元素 $A[i]$ 的排名。

1. for $i \leftarrow 1$ to n
2. $R[i] \leftarrow 0$ // 初始化
3. end for
4. for($i \leftarrow 2$ to n) // 逐对比较所有的元素
5. for $j \leftarrow 1$ to i
6. if ($A[j] < A[i]$) then $R[i] \leftarrow R[i] + 1$
7. else $R[j] \leftarrow R[j] + 1$
8. end if
9. end for
10. end for



例2：计算 $A[1..n]$ 中元素的排名

- 基本运算：元素的比较。
- 对每个 i 的取值，执行元素比较的次数是 i ，总的比较次数为 $2+\dots+n=(n-1)n/2$ 。
- for循环的额外开销、初始化数组 R 的开销、以及每次比较 A 中的两个数时对 R 元素进行的增值的计算开销都没有元素比较次数高。



回顾之前的算法例

- LINEARSEARCH: 基本操作: 元素比较
- BINARYSEARCH: 基本操作: 元素比较
- SELECTIONSORT: 基本操作: 元素比较
- INSERTIONSORT: 基本操作: 元素比较
- BOTTOMUPSORT: 基本操作: 元素比较

1.2.3 时间复杂性/时间复杂度

- 当我们去掉算法运行时间函数中的低阶项和首项常数，就表示我们正在度量算法的渐近运行时间。在算法分析中，术语“时间复杂度”或“时间复杂性”就用来表示该渐进运行时间。
- 为了形式化表示算法时间复杂性，广泛使用特殊的数学符号 O 、 Ω 、 Θ 等表示。这些符号可以方便通过最少的数学计算就能进行比较和分析算法的运行时间。

1.2.4 渐进表示

记：算法的计算时间为 $f(n)$

数量级限界函数为 $g(n)$

其中，

- n 是输入或输出规模的某种测度，即问题的规模。
- $f(n)$ 表示算法的“实际”执行时间。
- $g(n)$ 是形式简单的函数，如 n^m ， $\log n$ ， $2n$ ， $n!$ 等。是事前分析中通过对计算时间或频率计数统计分析所得的与机器及语言无关的函数。

渐进表示——大 O-记号 (渐近上界记号)

定义1.2 设 $f(n)$ 和 $g(n)$ 是从自然数到非负正数集上的两个函数。如果存在正常数 c 和自然数 n_0 , 使得

$$\forall n \geq n_0, \text{ 有 } f(n) \leq c g(n)$$

则称 $f(n)$ 是 $O(g(n))$ 的, 记为 $f(n)=O(g(n))$ 。

换句话说, 当 $\lim_{n \rightarrow \infty} f(n) / g(n)$ 存在, 那么,

如果有 $\lim_{n \rightarrow \infty} f(n) / g(n) \neq \infty$ 则表示 $f(n)=O(g(n))$

根据定义可知: 函数 $f(n)$ 至多是函数 $g(n)$ 的 c 倍, 除非 $n < n_0$ 。即是说, 当 n 充分大时, **函数 $f(n)$ 有上界**, 且 $g(n)$ 是它的一个上界函数 (在相差一个正常数倍的情况下)。**即当 n 充分大时, $f(n)$ 的阶不高于 $g(n)$ 的阶。**

渐进表示——大 O-记号 (渐近上界记号)

- 例：设 $f(n) = 10n^2 + 20n$. 由于当 $n \geq 1$, $f(n) \leq 30n^2$. 所以 $f(n) = O(n^2)$
换种方式, $\lim_{n \rightarrow \infty} f(n)/n^2 = 10 \neq \infty$. 所以 $f(n) = O(n^2)$
- $f(n) = O(1)$: $f(n)$ 等于非零常数的情形。
- $3n + 2 = O(n)$: 可取 $c = 4$, $n_0 = 2$ 。
- $100n + 6 = O(n)$: 可取 $c = 101$, $n_0 = 6$ 。
- $10n^2 + 4n + 3 = O(n^2)$: 可取 $c = 11$, $n_0 = 6$
- $6 \times 2^n + n^2 = O(2^n)$: 可取 $c = 7$, $n_0 = 4$.

渐进表示——大 O-记号 (渐近上界记号)

• 例：

- 因为 $\lim_{n \rightarrow \infty} \frac{3n+2}{n} = 3$, 所以 $3n+2=O(n)$;
- 因为 $\lim_{n \rightarrow \infty} \frac{10n^2+4n+2}{n^2} = 10$, 所以 $10n^2+4n+2=O(n^2)$;
- 因为 $\lim_{n \rightarrow \infty} \frac{6 \cdot 2^n + n^2}{2^n} = 6$, 所以 $6 \cdot 2^n + n^2 = O(2^n)$;
- 因为 $\lim_{n \rightarrow \infty} \frac{n^{16} + 3 \cdot n^2}{2^n} = 0$, 所以 $n^{16} + 3 \cdot n^2 = O(2^n)$;

$$n^3 + n^2 \log n = O(n^3);$$

$$n^4 + n^{2.5} \log^{20} n = O(n^4); \quad 2^n n^4 \log^3 n + 2^n n^5 / \log^3 n = O(2^n n^5).$$

渐进表示——大 O-记号 (渐近上界记号)

注意事项

- 1. 用来作比较的函数 $g(n)$ 应该尽量接近所考虑的函数 $f(n)$.
 $3n+2=O(n^2)$ 松散的界限; $3n+2=O(n)$ 较好的界限。
- 2. $f(n)=O(g(n))$ 不能写成 $g(n)=O(f(n))$, 因为两者并不等价。
实际上, 这里的等号并不是通常相等的含义。按照定义, 用集合符号更准确些。
- 3. $O(g(n))=\{f(n)|f(n)\text{满足: 存在正的常数}c\text{ 和}n_0, \text{使得当}n\geq n_0\text{ 时, }f(n)\leq cg(n)\}$ 。所以, 人们常常把 $f(n)=O(g(n))$ 读作: “ $f(n)$ 是 $g(n)$ 的一个大O 成员”。

渐进表示— Ω -记号 (渐近下界记号)

定义 1.3 设 $f(n)$ 和 $g(n)$ 是从自然数到非负正数集上的两个函数。如果存在正常数 c 和自然数 n_0 ，使得

$$\forall n \geq n_0, \text{ 有 } f(n) \geq cg(n)$$

则称 $f(n)$ 是 $\Omega(g(n))$ 的，记为 $f(n) = \Omega(g(n))$ 。

也就是说，当 $\lim_{n \rightarrow \infty} f(n)/g(n)$ 存在，且

$$\lim_{n \rightarrow \infty} f(n)/g(n) \neq 0 \text{ 则表示 } f(n) = \Omega(g(n))$$

- 根据定义可知：函数 f 至少是函数 g 的 c 倍，除非 $n < n_0$ 。即是说，当 n 充分大时， g 是 f 的一个下界函数（在相差一个正常数倍的情况下）。即当 n 充分大时， $f(n)$ 的阶不低于 $g(n)$ 的阶。

渐进表示— Ω -记号 (渐近下界记号)

- 例：

Let $f(n) = 10n^2 + 20n$. Then,

由于当 $n \geq 1$, $f(n) \geq n^2$. 因此有 $f(n) = \Omega(n^2)$

也有, $\lim_{n \rightarrow \infty} f(n)/n^2 = 10 \neq 0$.



渐进表示— Θ -记号（紧渐进界记号）

定义 1.4 如果存在2个正的常数 c_1, c_2 和自然数 n_0 ，使得

$$\forall n \geq n_0, \text{ 有 } c_1 g(n) \leq f(n) \leq c_2 g(n)$$

则称 $f(n)$ 是 $\Theta(g(n))$ ，记为 $f(n) = \Theta(g(n))$ 。

也就是说，当 $\lim_{n \rightarrow \infty} f(n) / g(n)$ 存在，并且 $\lim_{n \rightarrow \infty} f(n) / g(n) = c$ ，
则 $f(n) = \Theta(g(n))$ ， c 必须是大于 0 的常量。

- 根据定义可知：函数 f 介于函数 g 的 c_1 和 c_2 倍之间,即当 n 充分大时, g 既是 f 的下界,又是 f 的上界.
- 可以推断, $f(n) = \Theta(g(n))$ 当且仅当 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$ 。
此时称 $f(n)$ 与 $g(n)$ 同阶。

渐进表示— Θ -记号（紧渐进界记号）

• 例：

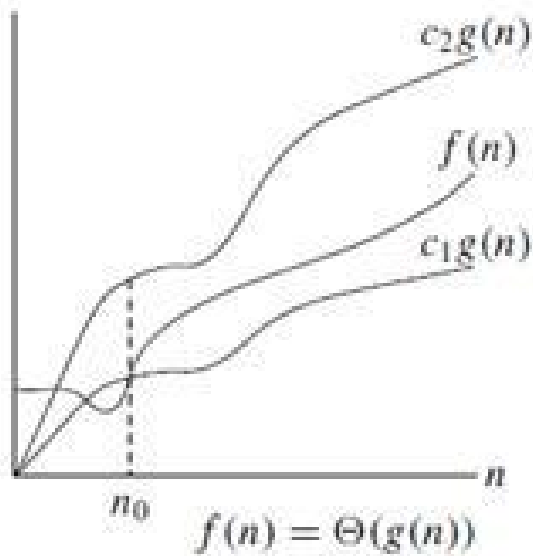
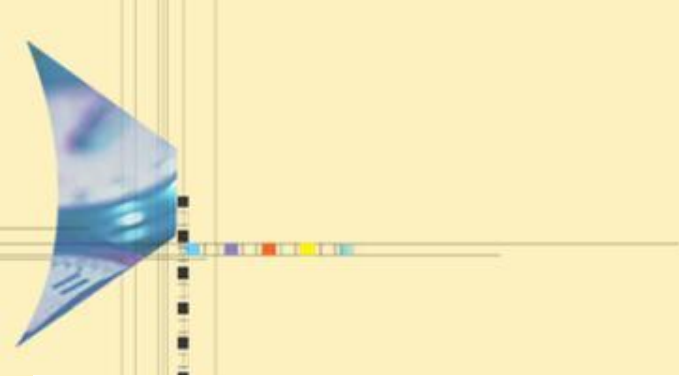
Let $f(n) = 10n^2 + 20n$.

由于当 $n \geq 1$, $n^2 \leq f(n) \leq 30n^2$. 因此有, $f(n) = \Theta(n^2)$.

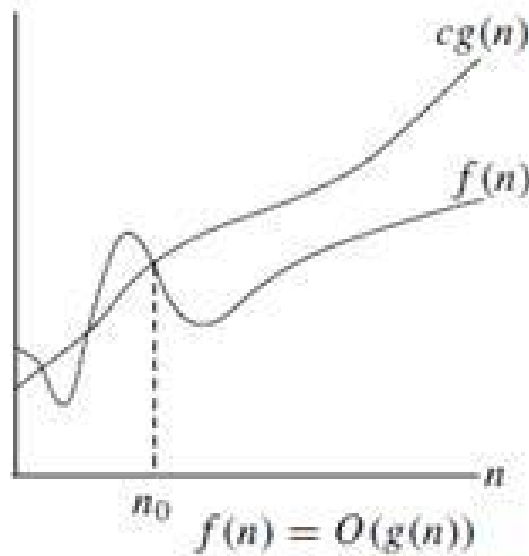
也有, $\lim_{n \rightarrow \infty} f(n)/n^2 = 10 > 0$.

- $3n + 2 = \Theta(n)$,
- $10n^2 + 4n + 2 = \Theta(n^2)$ 。
- $5 \times 2^n + n^2 = \Theta(2^n)$

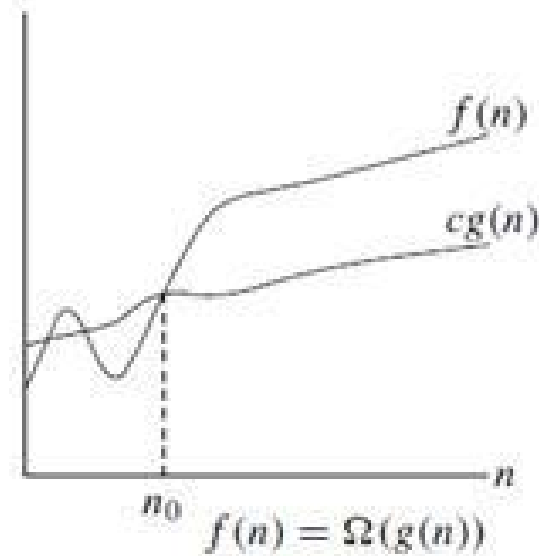




(a)



(b)



(c)



渐进表示—小o-记号 (非紧上界记号)

定义 1.5 对于任意给定的 $c > 0$ ，都存在正整数 n_0 ，使得当 $n \geq n_0$ 时有 $f(n) < cg(n)$ ，则称函数 $f(n)$ 是 $o(g(n))$ 的，记为 $f(n) = o(g(n))$ 。

也就是说，当 $\lim_{n \rightarrow \infty} f(n) / g(n)$ 存在，并且

$$\lim_{n \rightarrow \infty} f(n) / g(n) = 0 \text{ 则 } f(n) = o(g(n)).$$

- 可以推断： $f(n) = o(g(n))$ 成立前提：必须 满足 $f(n) = O(g(n))$ ，同时 满足 $g(n) \neq O(f(n))$ 。
- 根据定义，当 n 充分大时， $f(n)$ 的阶比 $g(n)$ 低。也说明，当 n 趋于无穷时， $f(n)$ 对于 $g(n)$ 可以忽略不计。

渐进表示—小 ω -记号（非紧下界记号）

定义 1.5 对于任意给定的 $c > 0$ ，都存在正整数 n_0 ，使得当 $n \geq n_0$ 时有 $f(n) > cg(n)$ ，则称函数 $f(n)$ 是 $\omega(g(n))$ 的，记为 $f(n) = \omega(g(n))$ 。

也就是说，当 $\lim_{n \rightarrow \infty} f(n) / g(n)$ 不存在，

$$\lim_{n \rightarrow \infty} f(n) / g(n) \rightarrow \infty \text{ 则 } f(n) = \omega(g(n))$$

根据定义，当 n 充分大时 $f(n)$ 的阶比 $g(n)$ 高，也说明，当 n 趋于无穷时， $g(n)$ 对于 $f(n)$ 可以忽略不计。

渐近记号在等式和不等式中的意义

- $f(n) = \Theta(g(n))$ 的确切意义是: $f(n) \in \Theta(g(n))$ 。
- 一般情况下, 等式和不等式中的渐近记号 $\Theta(g(n))$ 表示 $\Theta(g(n))$ 中的某个函数。

例如:

$2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ 表示

$2n^2 + 3n + 1 = 2n^2 + f(n)$, 其中 $f(n)$ 是 $\Theta(n)$ 中某个函数。

- 等式和不等式中的渐近记号 O , o , Ω 和 ω 的意义是类似的。

渐近记号的若干性质

(1) 传递性:

- $f(n) = \Theta(g(n)), g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n));$
- $f(n) = O(g(n)), g(n) = O(h(n)) \Rightarrow f(n) = O(h(n));$
- $f(n) = \Omega(g(n)), g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n));$
- $f(n) = o(g(n)), g(n) = o(h(n)) \Rightarrow f(n) = o(h(n));$
- $f(n) = \omega(g(n)), g(n) = \omega(h(n)) \Rightarrow f(n) = \omega(h(n));$

渐近记号的若干性质

(2) 反身性:

- $f(n) = \Theta(f(n))$;
- $f(n) = O(f(n))$;
- $f(n) = \Omega(f(n))$.

(3) 对称性:

- $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$.

(4) 互对称性:

- $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$;
- $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$;

渐近记号的若干性质

(5) 算术运算:

- $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$;
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$;
- $O(f(n)) * O(g(n)) = O(f(n) * g(n))$;
- $O(cf(n)) = O(f(n))$;
- $g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n))$ 。



常用阶的比较

n	1	$\lg n$	n	$n \lg n$	n^2	n^3	2^n
1	1	0.00	1	0	1	1	2
10	1	3.32	10	33	100	1000	1024
100	1	6.64	100	664	10,000	1,000,000	1.27×10^{30}
1000	1	9.97	1000	9970	1,000,000	10^9	1.07×10^{301}

多项式函数与指数函数

时间复杂度函数	问题规模					
	10	20	30	40	50	60
n	10^{-5}	$2*10^{-5}$	$3*10^{-5}$	$4*10^{-5}$	$5*10^{-5}$	$6*10^{-5}$
n^2	10^{-4}	$4*10^{-4}$	$9*10^{-4}$	$16*10^{-4}$	$25*10^{-4}$	$36*10^{-4}$
n^3	10^{-3}	$8*10^{-3}$	$27*10^{-3}$	$64*10^{-3}$	$125*10^{-3}$	$216*10^{-3}$
n^5	10^{-1}	3.2	24.3	1.7 分	5.2 分	13.0 分
2^n	.001 秒	1.0 秒	17.9 分	12.7 天	35.7 年	366 世纪
3^n	.059 秒	58 分	6.5 年	3855 世纪	$2*10^8$ 世纪	$1.3*10^{13}$ 世纪

表中默认单位为秒

多项式函数与指数函数

时间复杂度函数	1小时可解的问题实例的最大规模		
	计算机	快100倍的计算机	快1000倍的计算机
n	N_1	$100 N_1$	$1000 N_1$
n^2	N_2	$10 N_2$	$31.6 N_2$
n^3	N_3	$4.64 N_3$	$10 N_3$
n^5	N_4	$2.5 N_4$	$3.98 N_4$
2^n	N_5	$N_5 + 6.64$	$N_5 + 9.97$
3^n	N_6	$N_6 + 4.19$	$N_6 + 6.29$

多项式时间的算法

- 多项式时间的算法：
时间复杂度函数为 $O(p(n))$ 的算法，其中 $p(n)$ 是 n 的多项式
- 不是多项式时间的算法：
不存在多项式 $p(n)$ 使得该算法的时间复杂度为 $O(p(n))$ ，
包含指数时间甚至更高阶的算法

10亿次/秒机器求解的问题

- 快速排序算法给10万个数据排序, 运算量约为 $10^5 \times \log_2 10^5 \approx 1.7 \times 10^6$, 仅需 $1.7 \times 10^6 / 10^9 = 1.7 \times 10^{-3}$ 秒.
- Dijkstra算法求解1万个顶点的图的单源最短路径问题, 运算量约为 $(10^4)^2 = 10^8$, 约需 $10^8 / 10^9 = 0.1$ 秒.
- 回溯法解100个顶点的图的最大团问题, 运算量为 $100 \times 2^{100} \approx 1.8 \times 10^{32}$, 需要 $1.8 \times 10^{32} / 10^9 = 1.8 \times 10^{21}$ 秒
 $= 5.7 \times 10^{15}$ 年, 即5千7百万亿年!
- 1分钟能解多大的问题? 1分钟60秒, 这台计算机可做用快速排序算法可给 2×10^9 (即, 20亿)个数据排序; 用 Dijkstra算法可解 2.4×10^5 个顶点的图的单源最短路径问题; 而用回溯法一天只能解41个顶点的图的最大团问题。

1.2.5 空间复杂性/空间复杂度

为什么要考虑空间复杂性，如果程序实现，可能关注：

- 在多用户系统中运行时，需指明分配给该程序的内存大小；
- 想预先知道计算机系统是否有足够的内存来运行该程序；
- 一个问题可能有若干个不同的内存需求解决方案,从中择取；
- 用空间复杂性来估计一个程序可能解决的问题的最大规模。



1.2.5 空间复杂性/空间复杂度

- 一个算法所占用的存储空间，包括存储算法本身所占用的存储空间，算法的输入输出数据所占用的存储空间和算法在运行中临时使用的存储空间这三个方面。
- 算法运行中临时使用的空间是为了求解问题的实例而执行的计算步骤所需的内存空间。
- 将算法关于时间复杂性的定义和渐近界的讨论“移植到”对算法空间复杂性的讨论，即算法的空间复杂度(Space Complexity)是对一个算法在运行中临时使用存储空间的渐近度量，使用和时间复杂性度量相同的符号。

例：顺序查找算法的空间复杂性分析

算法1.1 LINEARSEARCH

输入: n 个元素的数组 $A[1 \cdots n]$ 和元素 x 。

输出: 如果 $x = A[j]$, $1 \leq j \leq n$, 则输出 j , 否则输出 0

1. $j \leftarrow 1$

2. **while** ($j < n$) **and** ($x \neq A[j]$)

3. $j \leftarrow j + 1$

4. **end while**

5. **if** $x = A[j]$ **then return** j **else return** 0.

参数 x , n , 局部变量 j , 输出常数 0 等是算法运行中临时需要使用存储空间来存储, 假定每个需要 2 个字节, 总共需要空间为 8 个字节, 其独立于 n 的常数, 所以算法的空间复杂性为 $\Theta(1)$ 。

1.2.6 最优算法

- 如果可以证明任何一个求解问题 Π 的算法时间复杂度必定是 $\Omega(f(n))$ ，那么我们把在 $O(f(n))$ 时间内求解问题 Π 的任何算法都称为问题 Π 的**最优算法**。
- 文献中被广泛使用的定义没有考虑空间复杂性，原因如下：
 - 1) 时间比空间珍贵；
 - 2) 大多数最优算法的空间复杂性同处 $O(n)$ 阶内。

1.2.7 不同情况下的算法时间复杂度

- 最好情况下的时间复杂度

算法求解输入规模为 n 的实例所需要的最少时间

$$B(n) = \min \{t_I | I \in S\}$$

- 最坏情况下的时间复杂度

算法求解输入规模为 n 的实例所需要的最长时间

$$W(n) = \max \{t_I | I \in S\}$$

- 平均情况下的时间复杂度

算法求解输入规模为 n 的实例所需要的平均时间

$$A(n) = \sum_{I \in S} t_I p_I$$

S : 实例集, t_I : 实例 I 的基本运算次数, p_I : I 出现的概率

例：顺序查找算法不同情况复杂性分析

- 最坏情况, $W(n) = \max \{ T_1 \mid \text{size}(I)=n \} = O(n)$
- 最好情况, $B(n) = \min \{ T_1 \mid \text{size}(I)=n \} = O(1)$
- 在平均情况下, 假设:
 - (a) 搜索成功的概率为 p ($0 \leq p \leq 1$);
 - (b) 在数组的每个位置 i ($0 \leq i < n$) 搜索成功的概率相同, 均为 p/n 。

$$\begin{aligned} A(n) &= \frac{p}{n} \sum_{i=1}^n i + n(1-p) = \frac{p(n+1)}{2} + n(1-p) \\ &= \left(1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + 3 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n} \right) + n \cdot (1-p) \end{aligned}$$

- 当 $p=1$, 则 $A(n) = (n+1)/2$; 当 $p=0$, 则 $A(n) = n$;